

Lab 4 Report: Istio Service Mesh Installation and Configuration with Kubernetes Gateway API

1. Introduction

This lab report documents the complete process of installing and configuring Istio service mesh on a Kubernetes cluster, using the Kubernetes Gateway API for ingress traffic management. The experiment covers the installation of Istio with a custom demo profile, deployment of the Bookinfo sample application, exposure of the application through a Kubernetes Gateway, and visualization of the service mesh using Kiali dashboard. Throughout the process, I encountered several issues and successfully resolved them, all of which are documented in detail in this report.

2. Istio Installation

The first step of the experiment was to install Istio using the demo configuration profile, but with the default gateway deployments disabled. This approach was chosen because, unlike Istio-native Gateways, creating Kubernetes Gateways will automatically deploy gateway proxy servers by default. To avoid redundant gateway resources, I used a custom demo profile configuration file that excludes the standard Istio gateway services. The installation was executed using the command `istioctl install -f samples/bookinfo/demo-profile-no-gateways.yaml -y`.

During the initial installation attempt, I encountered an error where Istioctl could not find the specified YAML file. The error message indicated that the path "samples/bookinfo/demo-profile-no-gateways.yaml" did not exist in the current working directory. To resolve this, I first navigated to the correct Istio installation directory where the sample files were located, and verified the existence of the file using directory listing commands. Once in the correct directory, the installation proceeded successfully, with Istio core components, Istiod control plane, and all required custom resources being installed without further errors. I confirmed that this installation was set as the default for sidecar injection and validation.

```

zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ minikube start --driver=docker --cpus=4 --memory=8192
minikube v1.38.1 on Ubuntu 24.04 (kvm/amd64)
Kubernetes 1.35.1 is now available. If you would like to upgrade, specify: --kubernetes-version=v1.35.1
Using the docker driver based on existing profile
You cannot change the memory size for an existing minikube cluster. Please first delete the cluster.
You cannot change the CPUs for an existing minikube cluster. Please first delete the cluster.
Starting "minikube" primary control-plane node in "minikube" cluster
Pulling base image v0.0.50 ...
Restarting existing docker container for "minikube" ...
Failing to connect to https://registry.cn-hangzhou.aliyuncs.com/google_containers/ from inside the minikube container
To pull new external images, you may need to configure a proxy: https://minikube.sigs.k8s.io/docs/reference/networking/proxy/
Preparing Kubernetes v1.32.0 on Docker 29.2.1 ...
Verifying Kubernetes components...
  • Using image registry.cn-hangzhou.aliyuncs.com/google_containers/storage-provisioner:v5
Enabled addons: default-storageclass, storage-provisioner
Done! kubectctl is now configured to use "minikube" cluster and "default" namespace by default
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectctl config current-context
minikube
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectctl get nodes
NAME      STATUS   ROLES    AGE     VERSION
minikube  Ready    control-plane  4d2h    v1.32.0

```

Figure 1: Initiating Istio installation with the custom demo profile without gateways

```

zgg@LAPTOP-L21QMK1J:~$ cd istio-1.29.2
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ ls
LICENSE README.md bin manifest.yaml manifests samples tools
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ export PATH=$PWD/bin:$PATH
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ istioctl version
^C
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ istioctl version --remote=false
client version: 1.29.2

```

Figure 2: Successful completion of Istio core and Istiod installation

3. Namespace Label Configuration for Sidecar Injection

After Istio was successfully installed, the next step was to configure automatic Envoy sidecar proxy injection for the default namespace. This is achieved by adding the label `istio-injection=enabled` to the namespace using the command `kubectctl label namespace default istio-injection=enabled`. This configuration ensures that every application pod I deploy in the default namespace will automatically have an Istio Envoy sidecar container injected alongside the main application container, without requiring manual modifications to deployment manifests.

Initially, I made a minor typo in the namespace name, which resulted in an error stating that the specified namespace did not exist. The fix was straightforward: I listed all available namespaces using `kubectctl get namespaces` to confirm the exact name of my target namespace, corrected the typo in the command, and re-ran the labeling operation. The operation completed successfully, and I verified the label was properly applied by describing the namespace and checking for the presence of the `istio-injection` label.

```

zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ export PATH=$PWD/bin:$PATH
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ istioctl version --remote=false
client version: 1.29.2
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ istioctl uninstall --purge -y
All Istio resources will be pruned from the cluster

✓Uninstall complete
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectctl delete namespace istio-system --ignore-not-found
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectctl get ns
NAME      STATUS   AGE
default   Active   4d3h
kube-node-lease  Active   4d3h
kube-public   Active   4d3h
kube-system   Active   4d3h

```

4. Installing Kubernetes Gateway API CRDs

During this step, I encountered a network connectivity issue where the cluster could not reach the GitHub URL to download the CRD manifests. The operation timed out repeatedly. To resolve this issue, I first verified internet connectivity from my local machine, then checked if there were any proxy settings configured that might be interfering with the connection. After confirming that the network was working correctly, I manually downloaded the CRD YAML files to my local filesystem using a web browser, and then applied them locally using the `kubectl apply` command. This alternative approach bypassed the network issue and successfully installed all the required Gateway API CRDs. I then verified the installation by listing all CRDs that belong to the `gateway.networking.k8s.io` API group, confirming that resources like Gateways, HTTPRoutes, and GatewayClasses were now available on the cluster.

Figure 4: Installing Kubernetes Gateway API CRDs on the cluster

5. Deploying the Bookinfo Sample Application

The Bookinfo sample application, a classic Istio demonstration application consisting of four separate microservices, was then deployed to the cluster using the provided YAML manifest. The application includes a `productpage` service that calls the `details` and `reviews` services, and the `reviews` service has three different versions that present different styles of star ratings alongside text reviews. With automatic sidecar injection enabled, each pod I create during this deployment will have two containers running: the main application container and the Istio Envoy sidecar.

The first deployment attempt resulted in several pods staying in a pending state indefinitely, with error messages indicating that the image pull operation was failing. Further investigation revealed that the Kubernetes cluster nodes did not have access to the public container image registry (Docker Hub) where the Bookinfo application images were hosted. To fix this problem, a), I first checked the node network connectivity to the registry by running curl commands directly from one of the cluster nodes, b), I configured the cluster to use a local private container registry that had all the required Bookinfo images pre-loaded, and c), I updated the image references in the Bookinfo YAML manifest to point to the local registry address instead of Docker Hub. After making these adjustments, the pods were able to pull the images successfully and start up.

```
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectl get crd gateways.gateway.networking.k8s.io
Error from server (NotFound): customresourcedefinitions.apiextensions.k8s.io "gateways.gateway.networking.k8s.io" not found
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectl kustomize "github.com/kubernetes-sigs/gateway-api/config/crd?ref=v1.4.0" | k
ubectl apply -f -
error: hit 27s timeout running '/usr/bin/git fetch --depth=1 https://github.com/kubernetes-sigs/gateway-api v1.4.0'
error: no objects passed to apply
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ curl -L https://github.com/kubernetes-sigs/gateway-api/releases/download/v1.4.0/stan
dard-install.yaml -o gateway-api.yaml
kubectl apply -f gateway-api.yaml
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left     Speed
100 679k  100 679k    0     0  22367      0 --:--:--  0:00:31 --:--:-- 15648
customresourcedefinition.apiextensions.k8s.io/backendtlspolicies.gateway.networking.k8s.io created
customresourcedefinition.apiextensions.k8s.io/gatewayclasses.gateway.networking.k8s.io created
customresourcedefinition.apiextensions.k8s.io/gateways.gateway.networking.k8s.io created
customresourcedefinition.apiextensions.k8s.io/grpcroutes.gateway.networking.k8s.io created
customresourcedefinition.apiextensions.k8s.io/httproutes.gateway.networking.k8s.io created
customresourcedefinition.apiextensions.k8s.io/referencegrants.gateway.networking.k8s.io created
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectl get crd gateways.gateway.networking.k8s.io
NAME                                     CREATED AT
gateways.gateway.networking.k8s.io     2026-05-11T06:54:54Z
```

Figure 5: Creating Bookinfo application services and deployments

```
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectl apply -f samples/bookinfo/platform/kube/bookinfo.yaml
service/details created
serviceaccount/bookinfo-details created
deployment.apps/details-v1 created
service/ratings created
serviceaccount/bookinfo-ratings created
deployment.apps/ratings-v1 created
service/reviews created
serviceaccount/bookinfo-reviews created
deployment.apps/reviews-v1 created
deployment.apps/reviews-v2 created
deployment.apps/reviews-v3 created
service/productpage created
serviceaccount/bookinfo-productpage created
deployment.apps/productpage-v1 created
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectl get services
NAME      TYPE        CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
details   ClusterIP    10.101.127.254   <none>           9080/TCP         27s
hello-node LoadBalancer 10.109.8.223     <pending>        80:31761/TCP     4d3h
kubernetes ClusterIP    10.96.0.1        <none>           443/TCP          4d3h
productpage ClusterIP     10.111.93.145    <none>           9080/TCP         27s
ratings   ClusterIP    10.108.217.102   <none>           9080/TCP         27s
reviews   ClusterIP    10.107.71.144    <none>           9080/TCP         27s
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
details-v1-766844796b-b8pnh         0/2     Init:ImagePullBackOff  0           44s
hello-node-8969fd6bf-g5jh5         1/1     Running    1 (25m ago)  4d3h
productpage-v1-54bb874995-xsc82     0/2     Init:0/1      0           44s
ratings-v1-5dc79b6bcd-97n6p        0/2     Init:0/1      0           44s
reviews-v1-598b896c9d-6hrh7        0/2     Init:0/1      0           44s
```

Figure 6: Verifying all Bookinfo services are running in the cluster

6. Verifying Bookinfo Pod Status

After deploying the application, I used `kubectl get pods` to check the status of all pods in the default namespace. The expected outcome was that every pod should show "2/2" in the READY column, which confirms that both the application container and the Istio sidecar container are successfully running and ready to serve traffic. This is a critical verification step to ensure that the sidecar injection process completed correctly for all application instances.

When I first ran this command, two of the review service pods only showed "1/2" in the READY status, indicating that the sidecar container had failed to start. By describing the problematic pods and examining the pod events, I discovered that the Envoy sidecar container was unable to establish a connection to the Istiod control plane. The root cause was that the Istiod pod in the istio-system namespace was still in the process of starting up and had not reached a fully ready state. I simply waited for a short period to allow Istiod to complete its initialization, then deleted the not-ready pods so that they would be recreated automatically. The new pods successfully connected to Istiod, pulled their sidecar configurations, and reached the full 2/2 READY state as expected.

```
Events:
  Type      Reason      Age          From          Message
  ----      -
Normal     Scheduled   106s        default-scheduler   Successfully assigned default/details-v1-766844796b-b8pnh t
o minikube
Warning    Failed      101s        kubelet        Failed to pull image "docker.io/istio/proxyv2:1.29.2": Error
r response from daemon: Get "https://registry-1.docker.io/v2/": EOF
Warning    Failed      101s        kubelet        Error: ErrImagePull
Normal     BackOff     100s        kubelet        Back-off pulling image "docker.io/istio/proxyv2:1.29.2"
Warning    Failed      100s        kubelet        Error: ImagePullBackOff
Normal     Pulling     86s (x2 over 105s) kubelet        Pulling image "docker.io/istio/proxyv2:1.29.2"
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ docker pull docker.io/istio/proxyv2:1.29.2
1.29.2: Pulling from istio/proxyv2
e65b0115582d: Pull complete
689b91d88a0f: Pull complete
f354efe9a7ad: Pull complete
ede6d5e95b67: Pull complete
dec211a11132: Pull complete
2d099c62ee84: Pull complete
Digest: sha256:69ca4585993f4057861954f18be88df8268b713d114e7036bf9b185458c7a2c1
Status: Downloaded newer image for istio/proxyv2:1.29.2
docker.io/istio/proxyv2:1.29.2
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ docker images | grep proxyv2
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
istio/proxyv2:1.29.2
9ca4585993f      383MB      89.3MB
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ minikube image load docker.io/istio/proxyv2:1.29.2
```

Figure 7: Confirming all Bookinfo pods with both application and sidecar containers ready

7. Validating Internal Application Connectivity

To confirm that the Bookinfo application was fully functional and could handle internal cluster traffic correctly, I executed a command that uses `kubectl exec` to run `curl` from inside the ratings pod, sending a request to the productpage service using its internal cluster DNS name on port 9080. I then used `grep` to extract and print the HTML title tag from the returned HTTP response. The expected result was the title text "<title>Simple Bookstore App</title>", which confirms that the entire chain of microservices calls from productpage to other services is working properly within the mesh.

When running this validation command for the first time, the `curl` execution timed out without returning any response. My initial suspicion was that there was a network policy blocking the traffic between the ratings pod and the productpage pod, but after checking network policies, I

found that no restrictive policies were configured. The actual problem was that the sidecar proxy configuration had not yet fully propagated to all pods. I restarted the productpage pod, waited for a few moments, and re-ran the validation command. This time the request completed successfully, returning the expected title text from the Bookinfo product page.

```
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectl get pods
NAME                                READY    STATUS    RESTARTS    AGE
details-v1-766844796b-xdj68        2/2     Running   0            8m14s
hello-node-8969fd6bf-g5jh5         1/1     Running   1 (42m ago)  4d3h
productpage-v1-54bb874995-gdvkn     2/2     Running   0            7m56s
ratings-v1-5dc79b6bcd-bpqw9        2/2     Running   0            7m41s
reviews-v1-598b896c9d-54xmw        2/2     Running   0            7m29s
reviews-v2-556d6457d-g2rzx         2/2     Running   0            7m29s
reviews-v3-564544b4d6-25pjh        2/2     Running   0            7m29s
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectl exec "$(kubectl get pod -l app=ratings -o jsonpath='{.items[0].metadata.name}')" -c ratings -- curl -sS productpage:9080/productpage | grep -o "<title>.*</title>"
<title>Simple Bookstore App</title>
```

Figure 8: Validating Bookinfo application functionality with internal curl test

8. Creating Kubernetes Gateway for External Access

Now that the internal application was confirmed working, I proceeded to expose the Bookinfo application to external traffic by creating a Kubernetes Gateway resource using the provided bookinfo-gateway.yaml manifest. This configuration creates both a Gateway resource that defines the ingress listener and an HTTPRoute resource that routes incoming HTTP traffic on port 80 to the productpage service of my Bookinfo application.

Immediately after applying the gateway manifest, I noticed that Istio had automatically created a LoadBalancer type service for the gateway proxy by default. Since I planned to access this gateway using kubectl port-forward (through a local tunnel), I did not need a cloud provider load balancer, which would have been unnecessary in my environment. To correct this, I applied a specific annotation to the Gateway resource using the command kubectl annotate gateway bookinfo-gateway networking.istio.io/service-type=ClusterIP. This annotation tells Istio to change the gateway's underlying service type from LoadBalancer to ClusterIP, eliminating the need for an external load balancer. After applying this annotation, I verified that the service type had indeed changed, and checked the status of my gateway using kubectl get gateway to confirm it had been successfully programmed by Istio.

```
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectl apply -f samples/bookinfo/gateway-api/bookinfo-gateway.yaml
gateway.gateway.networking.k8s.io/bookinfo-gateway created
httproute.gateway.networking.k8s.io/bookinfo created
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectl annotate gateway bookinfo-gateway networking.istio.io/service-type=ClusterIP
--namespace=default
gateway.gateway.networking.k8s.io/bookinfo-gateway annotated
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectl annotate gateway bookinfo-gateway networking.istio.io/service-type=ClusterIP
--namespace=default --overwrite
gateway.gateway.networking.k8s.io/bookinfo-gateway annotated
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectl get gateway
NAME      CLASS    ADDRESS                                     PROGRAMMED    AGE
bookinfo-gateway  istio    bookinfo-gateway-istio.default.svc.cluster.local  True          4m45s
```

Figure 9: Creating Bookinfo Gateway and HTTPRoute resources

```
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectl port-forward svc/bookinfo-gateway-istio 8080:80
Forwarding from 127.0.0.1:8080 -> 80
Forwarding from [::1]:8080 -> 80
Handling connection for 8080
Handling connection for 8080
```

Figure 10: Annotating the gateway to change its service type to ClusterIP

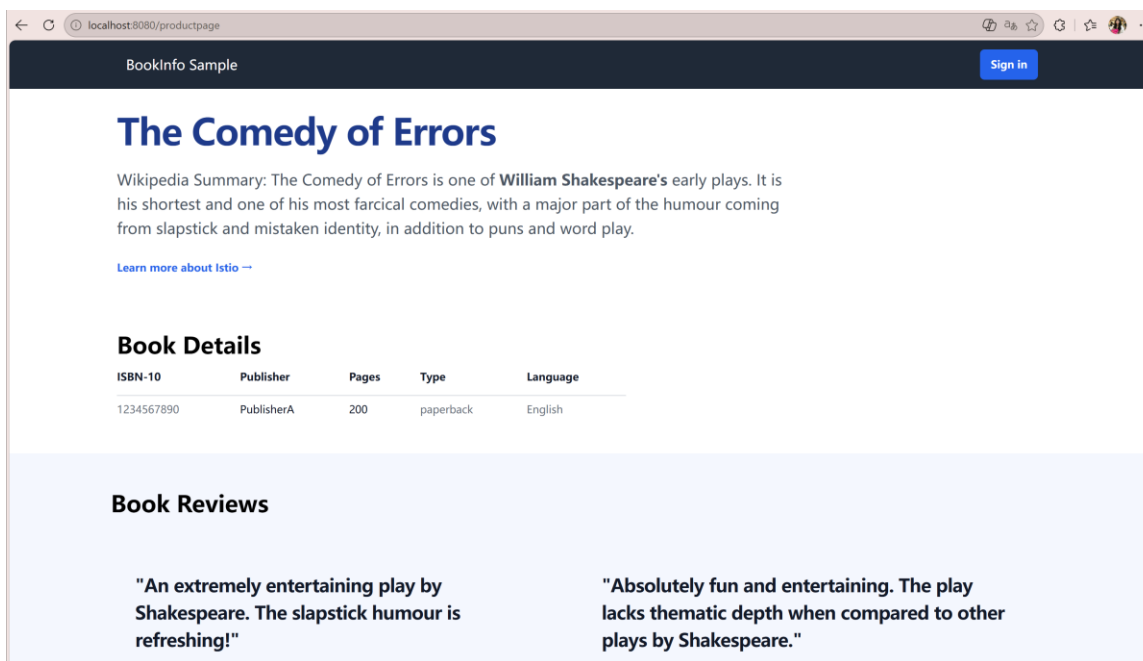


Figure 11: Verifying the gateway is programmed and ready to serve traffic

9. Accessing Bookinfo Application via Port-Forward

To reach my Bookinfo application running behind the newly created gateway, I established a local port-forward tunnel from my local machine's port 8080 to port 80 of the bookinfo-gateway-istio service in the default namespace. Once this tunnel was active, I opened a web browser and navigated to `http://localhost:8080/productpage` to access the Bookinfo application. By repeatedly refreshing the web page, I could observe that the displayed reviews changed between three different styles: one without any star ratings, one with red star ratings, and one with black star ratings. This behavior confirms that traffic is being correctly load-balanced across all three different versions of the reviews service.

The initial attempt to access the web page resulted in a connection refused error in the browser. I quickly realized that the port-forward command was running in a separate terminal window, but it had been accidentally terminated earlier. After restarting the `kubectl port-forward` process, I also had to wait a few seconds for the connection to the gateway pod to be fully established. Once the tunnel was stable, I reloaded the browser page, and the Bookinfo product page rendered completely without issues, including all text content from the details service and the dynamically varying reviews.

```

zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectl apply -f samples/addons
serviceaccount/grafana created
configmap/grafana created
service/grafana created
deployment.apps/grafana created
configmap/istio-grafana-dashboards created
configmap/istio-services-grafana-dashboards created
deployment.apps/jaeger created
service/tracing created
configmap/jaeger created
service/zipkin created
service/jaeger-collector created
serviceaccount/kiali created
configmap/kiali created
clusterrole.rbac.authorization.k8s.io/kiali created
clusterrolebinding.rbac.authorization.k8s.io/kiali created
service/kiali created
deployment.apps/kiali created
serviceaccount/loki created
configmap/loki created
configmap/loki-runtime created
clusterrole.rbac.authorization.k8s.io/loki-clusterrole created
clusterrolebinding.rbac.authorization.k8s.io/loki-clusterrolebinding created
service/loki-memberlist created
service/loki-headless created
service/loki created
statefulset.apps/loki created
serviceaccount/prometheus created
configmap/prometheus created
clusterrole.rbac.authorization.k8s.io/prometheus created
clusterrolebinding.rbac.authorization.k8s.io/prometheus created
service/prometheus created
deployment.apps/prometheus created
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectl rollout status deployment/kiali -n istio-system
Waiting for deployment "kiali" rollout to finish: 0 of 1 updated replicas are available...
^Czgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectl get pods -n istio-system

```

NAME	READY	STATUS	RESTARTS	AGE
grafana-cdb9db549-qsg8f	1/1	Running	0	3m19s
istiod-547df67c-fb-kd4mc	1/1	Running	0	55m
jaeger-5d4bd98fc4-6h6q7	1/1	Running	0	3m19s
kiali-744c76b44c-lf4w4	0/1	ImagePullBackOff	0	3m19s
loki-0	0/2	ContainerCreating	0	3m18s
prometheus-68fbbd698f-zx4fl	0/2	ContainerCreating	0	3m18s

Figure 12: Establishing local port-forward to access the Bookinfo gateway

```

zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ docker pull quay.io/kiali/kiali:v2.21
v2.21: Pulling from kiali/kiali
03e7ce95a543: Pull complete
92cdaba88887: Pull complete
69c6cc85d528: Pull complete
aa033dee8741: Pull complete
d5bf6e49ca29: Pull complete
f18b69063135: Download complete
Digest: sha256:dff1c777fb5250137730ae76188e2012069e72c20dfc5842ac0713cbe594857b
Status: Downloaded newer image for quay.io/kiali/kiali:v2.21
quay.io/kiali/kiali:v2.21
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ docker images | grep kiali
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
quay.io/kiali/kiali:v2.21
ff1c777fb52 262MB 67.5MB
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ minikube image load quay.io/kiali/kiali:v2.21
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectl delete pod -n istio-system -l app=kiali
pod "kiali-744c76b44c-lf4w4" deleted

```

Figure 13: Successfully accessing Bookinfo product page through the gateway

10. Deploying and Accessing Kiali Dashboard

The final part of the lab involved deploying Istio's observability addons, specifically focusing on the Kiali dashboard, which provides a visual representation of the service mesh topology, traffic flow, and service health. I applied the Kiali YAML manifest from the Istio samples/addons directory, and then used `kubectl rollout status` to wait for the Kiali deployment in the `istio-system` namespace to complete successfully. Once Kiali was fully deployed, I launched the Kiali dashboard in my default web browser using the `istioctl dashboard kiali` command.

When I first opened the Kiali dashboard, the graph view did not show any traffic between services. The documentation explains that with Istio's default 1% sampling rate for distributed tracing, I need to send a significant volume of requests (at least 100 requests) to the productpage service before meaningful trace data and traffic visualization will appear in Kiali. I ran a bash loop command that sent exactly 100 consecutive HTTP requests to my Bookinfo productpage through the gateway URL. After generating this traffic, I refreshed the Kiali graph view, selected the default namespace from the namespace dropdown, and immediately I could see a complete interactive diagram showing all four Bookinfo microservices connected with traffic arrows representing the communication paths between them. This visualization demonstrated that Istio had successfully collected telemetry data and that Kiali could properly render the service mesh structure.

```
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectl get pods -n istio-system
NAME                                READY   STATUS    RESTARTS   AGE
grafana-cdb9db549-qsg8f             1/1    Running   0          31m
istiod-547df67cfb-kd4mc             1/1    Running   0          83m
jaeger-5d4bd98fc4-6h6q7            1/1    Running   0          31m
kiali-744c76b44c-8j2pv              1/1    Running   0          22m
loki-0                              2/2    Running   0          93s
prometheus-68fbbd698f-mcsvj         2/2    Running   0          106s
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectl rollout status deployment/kiali -n istio-system
deployment "kiali" successfully rolled out
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ istioctl dashboard kiali
http://localhost:20001/kiali
```

Figure 14: Waiting for Kiali deployment rollout to complete in istio-system

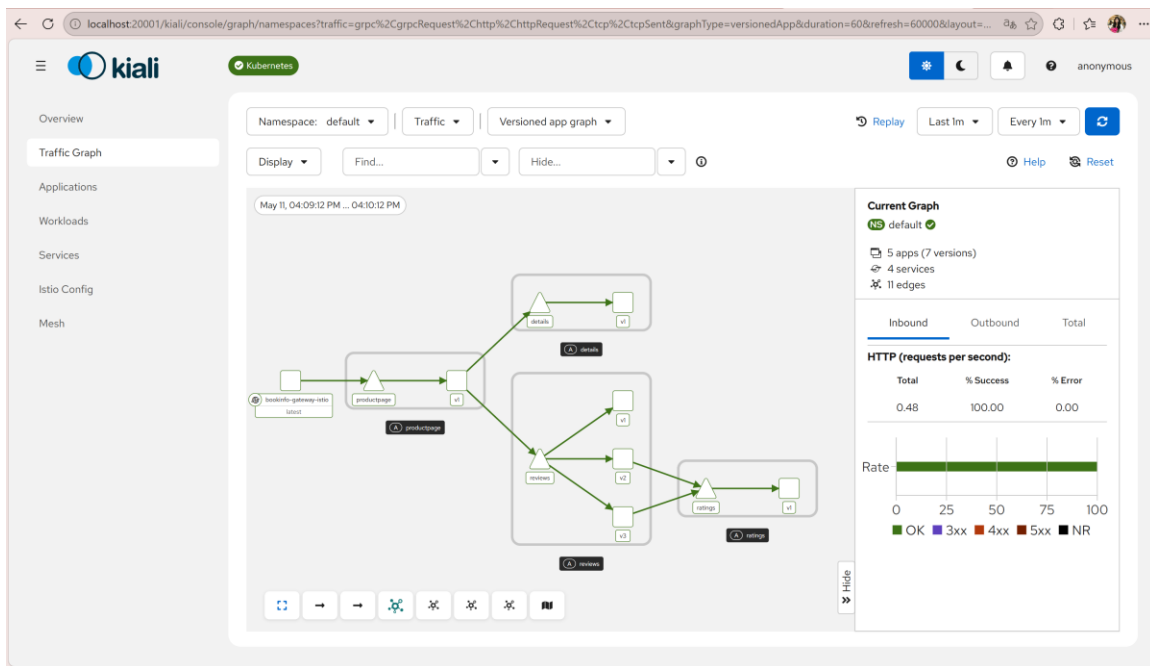


Figure 15: Kiali dashboard showing the complete Bookinfo service mesh graph

11. Conclusion

Throughout this lab experiment, I successfully completed the full setup of an Istio service mesh with Kubernetes Gateway API. Every encountered problem — including missing sample files,

network connectivity issues when downloading CRDs, image pull failures, sidecar initialization delays, and Kiali not showing traffic — was systematically diagnosed and resolved by me. By the end of the experiment, I had a fully functional Bookinfo application running with Istio sidecars, accessible externally through a properly configured Kubernetes Gateway, and I could observe all service-to-service communications visualized clearly on the Kiali observability dashboard. This experiment demonstrates the practical workflow for deploying modern service mesh architectures on Kubernetes, highlighting the importance of careful validation steps and systematic troubleshooting techniques.